

İSTİSNA YÖNETİMİ

Yapısal Programlamada Hata Yönetimi

Yapısal programlamada sorun olan, hata ayıklama kodu ile işlevsel kodun iç içe olması durumudur. Nesne yönelimli programlamanın getirdiği yenilikle bu sorun tarihe karışmıştır. Yapısal programlamada hatalı bir duruma yol açmamak için ya da hatalı bir durumla karşılaşıldığında programdan çıkılır. Aşağıda buna ilişkin bir C programı örneği verilmiştir;

```
#include <stdio.h>
int faktoriyel(int sayi) {
    if (sayi<0) return -1;
    if(sayi <= 1) return 1;
    return sayi * faktoriyel(sayi - 1);
}
int main() {
    int okunan,deger,hata;
    do {
        printf("Bir Sayı Giriniz:");
        scanf("%d",&okunan);
        deger=faktoriyel(okunan);
        printf("%d nin faktöriyeli: %d\n", okunan, deger);
        if (deger==-1) {
            hata=100;
            break; // exit(100);
        }
    } while (okunan!=0);
    return hata;
}
/* Programın Çıktısı:
Bir Sayı Giriniz:-1
-1 nin faktöriyeli: -1

...Program finished with exit code 100
*/
```

İstisna (Özel Durum) Yönetimi

Nesne Yönelimli Programlamada ise **istisna (özel durum)** (**exception**), bir programın yürütülmesi sırasında ortaya çıkan beklenmeyen bir sorundur. İstisna, programın **çalışması sırasında** (**run time**) oluşur. Beklenmeyen durumlarda imal edilen **istisna nesneleri** (**exception object**) vardır. İstisna iki türlü ortaya çıkar;

- **Eşzamanlı** (**synchronous**): Giriş verilerindeki bir hata nedeniyle bir şeylerin ters gitmesi veya programın çalıştığı mevcut veri türünü işleme durumunda oluşan istisnalar (örneğin bir sayıyı sıfıra bölmek).
- **Eşzamanlı olmayan** (**asynchronous**): Disk arızası, klavye kesintileri vb. gibi yazılan programın kontrolü dışındaki istisnalar.

İstisnai durumları yönetmek için **throw**, **try** ve **catch** anahtar kelimeleri kullanılır;

- **try** talimatı: İçerisinde istisnai bir duruma düşüleceğimizi belirten bir kod bloğunu temsil eder. Bu blokta bir veya daha fazla **catch** bloğu takip eder.
- **catch** talimatı: **try** bloğunda bir istisnai duruma düşüldüğünde yürütülecek bir kod bloğunu temsil eder. İstisnai işlemek için kullanılan kod, **catch** bloğunun içine yazılır.
- Bir istisnai duruma **throw** anahtar sözcüğü kullanılarak da düşülebilir. Bir program bir **throw** ifadesiyle karşılaştığında hemen içinde bulunduğu **try** bloğu dışına çıkılır ve ilgili istisnai işlemek için eşleşen bir **catch** bloğu bulmaya başlar.
- Bir **try** bloğu içinde birden fazla **throw** ifadesi bulunabilir.

```

try {
    /*
        ...
        İstisnai Durumun Ortaya Çıkabileceği Kod Bloğu...
        ...
    */
    throw SomeExceptionType("İstisnai Durum Açıklaması");
    /*
        throw an exception: istisnai bir duruma düşme
    */
} catch( Exception e ) {
    /*
        catch bloğu try bloğundaki istisnayı yakalamak ve gerekli işlemi yapmak için kullanılır.
    */
}

```

İstisnaları İşleme

Özel durumları işlemeye niçin ihtiyaç duyarız?

- Hata işleme kodunun işlevsel koddan (iş mantığını içeren koddan) ayrılması için.
- Yöntemler yalnızca seçtikleri istisnaları işleyebilir. Bir yöntemde birden çok istisnai duruma düşülebilir. Ancak bunlardan bazılarını **catch** bloğunda işlemeyi seçebilir. Geri kalanlarını yöntemi çağıran yere bırakabilir.
- Hata Türlerinin Gruplanması: C# dilinde yeni bir istisna sınıfı tanımlanabileceği gibi mevcut istisna sınıflarından da istisna nesnesi imal edilebilir.

Aşağıda dizi sınırları ile ilgili bir program örneği verilmiştir;

```

// See https://aka.ms/new-console-template for more information
int[] array = {10, 20, 30 };
try
{
    array[4]=40;
}
catch (Exception e)
{
    Console.WriteLine("Dizi indisi sınırların dışında!");
    Console.WriteLine($"Hata Mesajı:{e.Message}");
    Console.WriteLine($"Hata Verisi:{e.Data}");
    Console.WriteLine($"Hataya Sebep Olan İç Hata:{e.InnerException}");
}
/* Programın Çıktısı:
Dizi indisi sınırların dışında!
Hata Mesajı:Index was outside the bounds of the array.
Hata Verisi:System.Collections.ListDictionaryInternal
Hataya Sebep Olan İç Hata:

...Program finished with exit code 0
*/

```

Yukarıdaki örnekte **try** bloğunda izin verilmeyen indis istisnai bir durum fark ortaya çıkıyor ve altyapı bir istisna nesnesi imal ediliyor ve (istisna havuzuna) **fırlatılıyor** (**throw an exception**). **catch** bloğunda ise (havuzdan) **istisna nesnesi yakalanarak** (**catch an exception**) işleniyor. Örnekte standart **Exception** tipindeki **e** istisna nesnesi kullanılmıştır.

Ön Tanımlı İstisnalar

Ön tanımlı bazı istisna tipleri vardır ve aşağıdaki tabloda verilmiştir;

İstisna Tipi	Açıklama
System.ArithmeticException	ve System.OverflowException gibi System.DivideByZeroException aritmetik işlemler sırasında oluşan özel durumlar için bir temel sınıf.

İstisna Tipi	Açıklama
<code>System.ArrayTypeMismatchException</code>	Depolanmış öğenin türü dizinin türüyle uyumsuz olduğundan bir dizideki depo başarısız olduğunda oluşturulur.
<code>System.DivideByZeroException</code>	Bir sayı değerini sıfıra bölme girişi gerçekleştiğinde oluşturulur.
<code>System.IndexOutOfRangeException</code>	Sıfırdan küçük veya dizinin sınırlarının dışında olan bir dizin aracılığıyla bir diziye ekleme girişiminde bulunuldu.
<code>System.InvalidCastException</code>	Bir temel tür veya arabirimden türetilmiş bir türe açık dönüştürme çalışma zamanında başarısız olduğunda oluşturulur.
<code>System.NullReferenceException</code>	Referansın <code>null</code> , yani başvuru nesnenin gerekli olmasına neden olacak şekilde kullanıldığında oluşturulur.
<code>System.OutOfMemoryException</code>	Bellek ayırma girişi (aracılığıyla <code>new</code>) başarısız olduğunda oluşturulur.
<code>System.OverflowException</code>	Bir bağlam içindeki <code>checked</code> aritmetik işlem taşıdığına oluşturulur.
<code>System.StackOverflowException</code>	Yürütme yığını çok fazla bekleyen çağrıya sahip olarak tükendiğinde oluşturulur; genellikle çok derin veya ilişkisiz özyinelemenin göstergesidir.
<code>System.TypeInitializationException</code>	<code>static</code> bir alan/değişken <code>başlatıcı</code> (<code>initialize</code>) veya statik alan yapıcı bir istisna oluşturur ve onu yakalamak için kullanılır

Tablo 21. İstisna (Özel Durum) Tipleri

İstisnai Durumun Belirlenmesi

Sistem tarafından istisnaların yakalanması yerine istisnaları biz `throw` anahtar kelimesiyle yeni bir istisna nesnesi imal edip yönetebiliriz. Anı örnek bu durumda aşağıdaki gibi yazılabilir;

```
// See https://aka.ms/new-console-template for more information
int[] array = {10, 20, 30 };
try
{
    int indis = 4;
    if (indis >= array.Length) // İstisnai durum
    {
        //İstisna nesnesi oluşturulup (istisna havuzuna) fırlatılıyor
        throw new IndexOutOfRangeException("Dizi indisi sınırların dışında!");
    }
    array[indis]=40;
}
catch (Exception e)
{
    Console.WriteLine($"Hata Mesajı:{e.Message}");
    Console.WriteLine($"Hata Verisi:{e.Data}");
    Console.WriteLine($"Hataya Sebep Olan İç Hata:{e.InnerException}");
}
```

Bir `try` bloğunda birden fazla istisna nesnesi (istisna havuzuna) fırlatılabilir; Aşağıdaki örnekte `try` bloğunda birden fazla istisnai nesnesi fırlatılmıştır.

```
// See https://aka.ms/new-console-template for more information
int[] array = {10, 20, 30 };
try
{
    int indis = 2;
    if (indis >= array.Length) //koşul sağlanmaz!
    {
        throw new IndexOutOfRangeException("Dizi indisi sınırların dışında!");
    }
}
```

```

    array[2]=40;
    int bolen=0, bolunen=100;
    if (bolen == 0)
    {
        throw new DivideByZeroException("Bölme işlemi sıfıra bölünemez!");
    }
}
catch (IndexOutOfRangeException e)
{
    Console.WriteLine($"Hata Mesajı:{e.Message}");
    Console.WriteLine("İndis Hatası Olunca Neler Yapılmalı? Burada Yazılır.");
}
catch (DivideByZeroException e)
{
    Console.WriteLine($"Hata Mesajı:{e.Message}");
    Console.WriteLine("Sıfıra Bölme Olunca Neler Yapılmalı? Burada Yazılır.");
}
/*Program Çıktısı:
Hata Mesajı:Bölme işlemi sıfıra bölünemez!
Sıfıra Bölme Olunca Neler Yapılmalı? Burada Yazılır.
*/

```

Programda ilk karşılaşılan özel durum sıfıra bölme durumudur ve `throw` ile (istisna havuzuna) `DivideByZeroException("Bölme işlemi sıfıra bölünemez!");` nesnesi fırlatılır. Bu durumda `catch` bloklarına sırasıyla bakılır ilgili özel durum nesnesini işleyecek olan `catch` bloğu icra edilir.

when Süzgeci

Bazı durumlarda `when` süzgeci kullanılabilir;

```

// See https://aka.ms/new-console-template for more information
int[] array = { 10, 20, 30 };
try
{
    int indis = 2;
    if (indis >= array.Length) //koşul sağlanmaz!
    {
        throw new IndexOutOfRangeException("Dizi indisi sınırların dışında!");
    }
    array[2] = 40;
    int bolen = 0, bolunen = 100;
    if (bolen == 0)
    {
        throw new DivideByZeroException("Bölme işlemi sıfıra bölünemez!");
    }
}
catch (Exception e) when (e is IndexOutOfRangeException || e is DivideByZeroException)
{
    Console.WriteLine($"Hata Mesajı:{e.Message}");
    Console.WriteLine("Hata Olunca Neler Yapılmalı? Burada Yazılır.");
}

```

Kullanıcı Tanımlı İstisna Sınıfları

Aşağıda Kişi sınıfı yaş özelliğine ilişkin istisna içeren bir kod örneği verilmiştir;

```

// See https://aka.ms/new-console-template for more information
using System.Diagnostics; // StackTrace sınıfı için gerekli
int[] array = { 10, 20, 30 };
try
{
    int indis = 4;
    if (indis >= array.Length)
    {

```

```

        //throw new İndisİstisnası(); //Bu satır yada aşağıdaki
        string currentFile = new StackTrace(true).GetFrame(0).GetFileName();
        int currentLine = new StackTrace(true).GetFrame(0).GetFileLineNumber();
        throw new Dosyaİsimliİndisİstisnası(currentFile, currentLine);
    }
    array[indis] = 40;
}
catch (İndisİstisnası e)
{
    Console.WriteLine($"Hata Mesajı:{e.Message}");
    Console.WriteLine("İndis Hatası Olunca Neler Yapılmalı? Burada Yazılır.");
}
catch (Dosyaİsimliİndisİstisnası e)
{
    Console.WriteLine($"Hata Mesajı:{e.Message}");
    // Hata mesajı: D:\CSharp\Samples\Program.cs Dosyası 12 Satırında İndis Hatası!
    Console.WriteLine($"Hata Mesajı:{e.FileName}");
    // Hata mesajı: D:\CSharp\Samples\Program.cs
    Console.WriteLine($"Hata Mesajı:{e.LineNumber}");
    // Hata mesajı: 12
}

public class İndisİstisnası : Exception
{
    public İndisİstisnası() : base("Dizide olmayan bir elemana indis belirlendi!")
    { }
}

public class Dosyaİsimliİndisİstisnası : Exception
{
    public Dosyaİsimliİndisİstisnası(string fileName, int lineNumber) :
        base($"{fileName} Dosyası {lineNumber} Satırında İndis Hatası!")
    {
        FileName = fileName;
        LineNumber = lineNumber;
    }
    public string FileName { get; private set; }
    public int LineNumber { get; private set; }
}

```

finally Bloğu

finally bloğu, bir **try** blokta gerçekleştirilen eylemleri temizlemenizi sağlar. Varsa, **finally** bloğu en son bloktur ve eşleşen **catch** bloklardan sonra icra edilir. Özel bir durum/istisna oluşturma veya özel durum/istisna tipiyle eşleşen bir **catch** blok bulunması fark etmeksizin **finally** bloğu her zaman çalışır.

```

try
{
    /* istisna olabilecek kod burada yazılır. */
}
catch (Exception e)
{
    /* istisnayı yönetme kodu buraya yazılır. */
}
finally
{
    /* Bir istisna atılmış/yakalanmış olsun veya olmasın, yürütülecek kod buraya yazılır. */
}

```

catch bloğu programcı tarafından kodlanmayabilir. Dosyalarla ilgili bir örnek aşağıda verilmiştir;

```

FileStream f = null;
try
{

```

```
f = File.OpenRead("file.txt");
}
finally
{
    f?.Close(); /* f null olabilir, bu nedenle null koşullu işlecini kullanın.
                Çünkü dosya bir şekilde açılmamış olabilir. */
}
```

İstisnayı Tekrar Fırlatma

Bazı durumlarda istisna nesnesinin yakalanıp **catch** bloğunda işlenmesi sırasında mevcut istisna nesnesini **throw** ile (istisna havuzuna) **tekrar fırlatmak** (**re-throw**) gerekir.

```
try
{
    //...
}
catch (Exception ex)
{
    //...
    throw;
    //yada throw ex;
    //derleyiciye icra akışının asla try bloğundan ayrılmadığını bildirmek için yapılır.
}
```

Katmanlı mimaride bir katmandan diğerine istisnaları iletmek için bu yöntem kullanılır. Genellikle iç katmanlarda hatanın gösterileceği bir kullanıcı ara yüzü bulunmaz.

Özel Durum İşleme Kuralları

Kontrol Akışını İstisna Kodları ile Gerçekleştirmeyin

İstisna/özel durumları işleyen kod ile iş mantığını içeren kod nesne yönelimli paradigmada her zaman ayrı olmalıdır! Bunun için;

- İş mantığını içeren **kontrol akışı** (**control flow**) istisna kodlarıyla yapılmamalıdır. Bunun yerine **kontrol işlemleri** (**control operations**) ile iş mantığı gerçekleştirilmelidir.
- Ayrıca Bir kontrol akışı, if-else ifadeleriyle açıkça yapılabiliriyorsa, okunabilirliği ve performansı azalttığı için istisnalar kullanılmaz.

Aşağıdaki örneği göz önüne alalım;

```
//Bunu YAPMA!
object myObject;
void NesneYaz()
{
    Console.WriteLine(myObject.ToString());
}
```

Bu örnekle **Console.WriteLine** satırı icra edilmeye çalışıldığında **NullReferenceException** istisnası oluşur. Bunu yönetmek için aşağıdaki gibi bir kod yazmamalıyız;

```
// Bunu YAPMA!
object myObject;
void NesneYaz()
{
    try
    {
        Console.WriteLine(myObject.ToString());
    }
    catch(NullReferenceException ex)
    {
        // eğer object'in yeni bir örneğini oluşturup myObject'e atarsam sorunu çözebilirim.
        myObject = new object();
        // artık myObject ile çalışmaya devam edebilirim
    }
}
```

```
NesneYaz();
}
}
```

Doğru olan kodlama aşağıdaki gibi olmalıdır;

```
// Bunu YAP!
object myObject;
void NesneYaz()
{
    if(myObject == null)
        myObject = new object();
    // Programın icrası bu noktaya ulaştığında, myObject'in null olmadığından emin oluruz
    NesneYaz();
}
```

Zorunlu Olmadıkça İstisnayı Tekrar İmal Etmeyin

İstisnaları tekrar imal etmek (re-throw) pahalıdır. Performansı olumsuz etkiler! Rutin olarak başarısız olan kodlar için, performans sorunlarını en aza indirmek için **tasarım desenlerini** (design pattern) kullanabilirsiniz. Microsoft sitesinde yer alan İstisnalar ve Performans konusu istisnaların performansı önemli ölçüde etkileyebileceği zamanlarda faydalı olan iki tasarım desenini açıklamaktadır.²¹

İz Kaydı Olmadan İstisnaları Görmezden Gelmeyin

Aşağıdaki gibi bir istisna işleme kötü bir uygulamadır;

```
try
{
    //...
}
catch(Exception ex) { } // Boş catch bloğu kötü uygulamadır.
```

İstisnaları görmezden gelmek o anı kurtaracaktır ancak daha sonra sürdürülebilirlik için bir kaos yaratacaktır. İstisnaların **iz kaydını** (log) günlüğe kaydederken, istisna örneğini her zaman kaydetmelisiniz. Böylece tüm yığın izi günlüğe kaydedilir ve yalnızca istisna mesajı kaydedilmez! Aşağıdaki gibi bir kod daha doğru olabilir;

```
try
{
    //...
}
catch(NullException ex)
    LogManager.Log(ex.ToString());
}
```

İşleyemeyeceğiniz İstisnaları Yakalamayın

İşleyemeyeceğiniz istisnaları yakalamayın. Sadece yakalamayın. Yalnızca o konumda işleyebildiğiniz bir istisnayı yakalamalısınız. Bunun yerine, iyi bir çözümünüz olana veya ana fonksiyona ulaşana kadar bunların çağrı ağacında yukarı doğru ilerlemesine izin verin, bu durumda hata bilgilerinin doğru şekilde yönlendirildiğinden emin olabilirsiniz.

²¹ [https://learn.microsoft.com/en-us/previous-versions/dotnet/netframework-4.0/ms229009\(v=vs.100\)?redirectedfrom=MSDN](https://learn.microsoft.com/en-us/previous-versions/dotnet/netframework-4.0/ms229009(v=vs.100)?redirectedfrom=MSDN)